CMSC 420: B-Trees

Justin Wyss-Gallifent

January 16, 2026

1 Problems **2**

1 Problems

1. Consider this B-tree with $m = 5$:

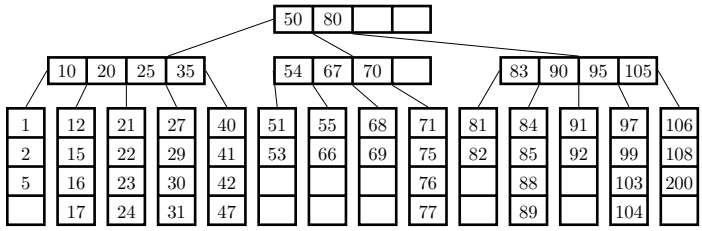

Figure 1: A B-Tree

Explain in words how the tree will be rebalanced if 13 is inserted. Use words and perhaps small sub-trees of the picture above. Don't redraw the entire tree, that would be insane.

Solution:
If 13 is inserted then the $12, 15, 16, 17$ node will become $12, 13, 15, 16, 17$ and will be overfull. We perform a left rotation since the left adjacent sibling has room.
The 12 will move to the parent and replace the 10 which will then move to the left adjacent sibling which then becomes $1, 2, 5, 10$.

2. Consider this B-tree with $m = 5$:

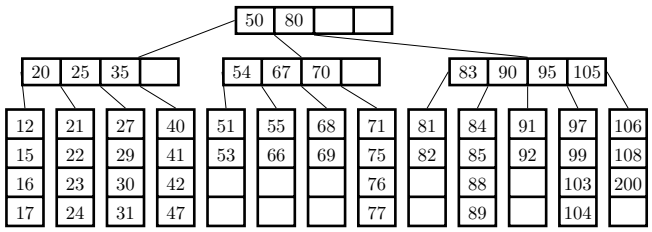

Figure 2: A B-Tree

Explain in words how the tree will be rebalanced if 28 is inserted. Use words and perhaps small sub-trees of the picture above. Don't redraw the entire tree, that would be insane.

Solution:
If 28 is inserted then the $27, 29, 30, 31$ node will become $27, 28, 29, 30, 31$ and will be overfull. We cannot rotate so we must split.
The 29 will promote to the parent $20, 25, 35$ which becomes $20, 25, 29, 35$. The left child of 29 will be $27, 28$ while the right child of 29 will be $30, 31$.

3. Consider this B-tree with $m = 5$:

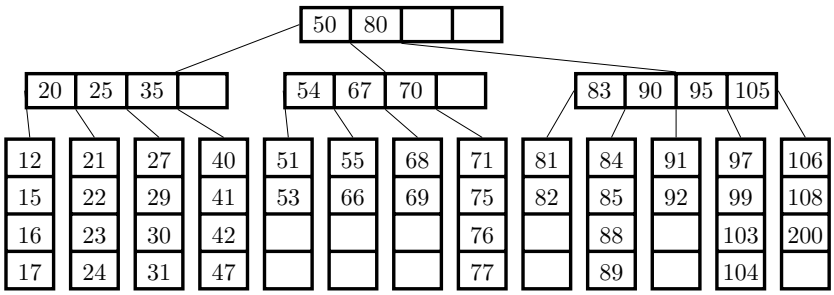

Figure 3: A B-Tree

Explain in words how the tree will be rebalanced if 51 is deleted. Use words and perhaps small sub-trees of the picture above. Don't redraw the entire tree, that would be insane.

Solution:
If 51 is deleted then the node containing just 53 is underfull. We cannot rotate with a sibling since since there is just one sibling and it doesn't have an extra key. Thus we have to merge with that sibling.
To merge we take 54 from the parent and merge to form the single child containing $51, 54, 55, 66$.
The parent still has two keys $67, 70$ (three children) and is fine.

4. Suppose a B-tree of order $m = 10$ has 10^j keys. For each of $j = 1, 3, 6, 9$ calculate the maximum and minimum possible heights.

Solution:
In general for j we have $k = 10^j$ keys and:

$$h \geq \log_{10}(10^j + 1) - 1$$
$$h \leq \log_5\left(\frac{10^j + 1}{2}\right)$$

For $j = 1$ we have:

$$h \geq \log_{10}(11) - 1 \approx 0.04139268515822492$$
$$h \leq \log_5\left(\frac{11}{2}\right) \approx 1.05921954433$$

So the minimum is 1 and the maximum is 1.
For $j = 3$ we have:

$$h \geq \log_{10}(1001) - 1 \approx 2.000434077479319$$
$$h \leq \log_5\left(\frac{1001}{2}\right) \approx 3.8619741406208$$

So the minimum is 3 and the maximum is 3.
For $j = 6$ we have:

$$h \geq \log_{10}(1000001) - 1 \approx 5.0000004342942646$$
$$h \leq \log_5\left(\frac{1000001}{2}\right) \approx 8.1533834117$$

So the minimum is 6 and the maximum is 8.
For $j = 9$ we have:

$$h \geq \log_{10}(1000000001) - 1 \approx 8.000000000434294$$
$$h \leq \log_5\left(\frac{1000000001}{2}\right) \approx 12.445412465208$$

So the minimum is 9 and the maximum is 12.
The range is growing by 1 each time.

5. Prove the unproven theorem from class: A B-tree of order m with k keys and height h satisfies:

   (a) The densest possible such tree has:

   $$k = m^{h+1} - 1$$

   Solution:
   The densest possible tree has m children per node and $m-1$ keys per node. Consequently the total number of keys in each level $0, 1, ..., h$ is:

Level	Max Nodes	Max Keys
0	1	$1(m-1)$
1	m	$m(m-1)$
2	m^2	$m^2(m-1)$
$\vdots$	$\vdots$	$\vdots$
h	m^h	$m^h(m-1)$

   Consequently the total number of keys is:

   $$\begin{aligned} k &= \sum_{i=0}^{h} m^i(m-1) \\ &= (m-1)\left(\frac{m^{h+1}-1}{m-1}\right) \\ &= m^{h+1} - 1 \end{aligned}$$

   (b) Consequently any such tree has:

   $$k \leq m^{h+1} - 1$$

   Solution:
   This follows immediately since the above count is the densest.

   (c) And it then follows that:

   $$h \geq \log_m(k+1) - 1$$

   Solution:
   We have:

   $$\begin{aligned} m^{h+1} - 1 &\geq k \\ m^{h+1} &\geq k+1 \\ h+1 &\geq \log_m(k+1) \\ h &\geq \log_m(k+1) - 1 \end{aligned}$$

   (d) Thus:

   $$h = \Omega(\lg k)$$

   Solution:
   This follows immediately.

6. Technically speaking when searching in a B-tree we need to search the keys of each node in order to figure out which child branch to follow.

   (a) Why is this not included into our asymptotic search time complexity?

   Solution:
   Because our asymptotic search time complexity is a function of k and m is assumed to be a constant Consequently the number of keys in a node is bounded by the constant $m-1$ and hence doesn't come into play.

   (b) Suppose we wanted to compute the asymptotic search time complexity not just as a function of k but as a function of both k and m. First do so, then use some basic precalculus to simplify this to a somewhat surprising result.

   Solution:
   Observe first that the height is $\mathcal{O}(\log_m k)$ and not simply $\mathcal{O}(\lg k)$. Then consider that binary search on the keys would be $\mathcal{O}(\lg(m-1)) = \mathcal{O}(\lg m)$ and so we would implement a $\mathcal{O}(\lg m)$ process a a total of $\mathcal{O}(\log_m k)$ times for a total of:

$$\mathcal{O}(\lg m \log_m k)$$

   However note that with the change of base formula for logarithms:

$$\mathcal{O}(\lg m \log_m k) = \mathcal{O}\left(\lg m \cdot \frac{\lg k}{\lg m}\right) = \mathcal{O}(\lg k)$$

   It might surprise you that the result has no m but the reality of the situation is that as m grows, the height of the tree shrinks and compensates, resulting in only k mattering.

7. Why is it not sensible to define a B-tree of order $m = 2$?

   Solution:
   Each node which is not the root would have either 0 or between $\lceil 2/2 \rceil = 1$ and 2 children. But if it had 1 child it must have $1 - 1 = 0$ keys, which isn't really sensible.

8. Suppose we have a B-tree with order $m \geq 3$. Suppose a non-root node is underfull and neither of its siblings is available for rotation. Prove that merging with either sibling does not result in an overfull node.

   Solution:
   Both siblings will have $\lceil m/2 \rceil - 1$ keys and together with the node in question there will be $\lceil m/2 \rceil - 2 + \lceil m/2 \rceil - 1 = 2\lceil m/2 \rceil - 3$ keys.
   We showed on the exam that this is at most $m - 2$ so once we pull a key down from the parent we will have at most $m - 1$ keys and the node will not be overfull.

9. Generally when we deal with an overfull or underfull node in a B-tree we attempt to rotate before splitting (for insertion) or merging (for deletion). Suppose we accidentally write our code so the splitting or merging is attempted first. What long-term effects might this have on the structure of the B-tree? Explain. This does not need to be justified rigorously, just explain what you think as best you can.

   Solution:
   Answers may vary but the general gist is as follows:
   With insertion if every overfull node is split we will end up with lots of nodes with very few keys if a great deal of insertion happens. In addition we will make more recursive calls which will take longer.
   With deletion if every underfull node is merged when possible this might actually be better than rotation since we will produce fewer nodes if a a great deal of deletion happens. However we will make more recursive calls which will take longer.

10. Suppose we have a B-tree of even order $m \geq 4$. Suppose $b > 1$ and the number of keys at time t is $k(t) = b^t$. Prove mathematically (think derivative) that as $t \to \infty$ the maximum height approaches a line and find the slope of that line.

Solution:
We have:

$$\begin{aligned}
h(t) &= \log_{\lceil m/2 \rceil}\left(\frac{b^t+1}{2}\right) \\
&= \log_{m/2}\left(\frac{b^t+1}{2}\right) \\
&= \log_{m/2}(b^t+1) - \log_{m/2} 2 \\
&= \frac{\ln(b^t+1)}{\ln(m/2)} - \log_{m/2} 2 \\
h'(t) &= \frac{b^t \ln b}{b^t+1}\frac{1}{\ln(m/2)} - 0 \\
&= \frac{b^t}{b^t+1}\frac{\ln b}{\ln(m/2)}
\end{aligned}$$

As $t \to \infty$ the first fraction approaches 1 so the slope approaches the second expression which is in fact:

$$\log_{m/2} b$$

11. Suppose a B-tree of order 10 contains 1999 keys. What are its maximum and minimum possible heights? You should be able to give exact integer answers (without a calculator).

Solution:
We have:

$$h \leq \log_{\lceil m/2 \rceil} \left(\frac{k+1}{2} \right) = \log_5 1000$$

Since h is an integer and $4 < \log_5 1000 < 5$ we have $h \leq 4$.
And we have:

$$h \geq \log_m (k+1) - 1 = \log_{10} 2000$$

Since h is an integer and $3 < \log_{10} 2000 < 4$ we have $h \geq 3$. Thus the height could only be 3 or 4.

12. Suppose a B-tree of order $m = 10$ contains exactly 9999 keys. For each of the following you should be able to get an exact answer without a calculator.

    (a) What is the minimum possible height?

    Solution:
    We have:

    $$h \geq \log_{10}(9999 + 1) - 1 = \log_{10}(10000) - 1 = 3$$

    (b) What is the maximum possible height?

    Solution:
    We have:

    $$h \leq \log_5((9999 + 1)/2) = \log_5(5000)$$

    Since $5^5 = 3125$ and $5^6 > 5000$ we know $\log_5(5000) < 6$ and so $h \leq 5$.

13. Suppose we have a B-tree with order $m \geq 3$. You may assume that m is odd. Prove that when a node is split that the two resulting new nodes are not underfull.

Solution in Terms of Children:
When we split a node it is because the node has $m+1$ children instead of its maximum of m.
Since m is odd we promote the median child, leaving $m-1$ which means each resulting new node has $(m-1)/2$ keys and each resulting new node has $(m-1)/2+1 = (m+1)/2$ children.
We claim that:

$$\frac{m+1}{2} \geq \left\lceil \frac{m}{2} \right\rceil$$

However since m is odd we have:

$$\left\lceil \frac{m}{2} \right\rceil = \frac{m+1}{2}$$

Thus the claim is satisfied.

Solution in terms of Keys:
When we split a node it is because the node has m keys instead of its maximum of $m-1$.
Since m is odd we promote the median, leaving $m-1$ which means each resulting new node has $(m-1)/2$ keys.
We claim that:

$$\frac{m-1}{2} \geq \left\lceil \frac{m}{2} \right\rceil - 1$$

However since m is odd we have:

$$\left\lceil \frac{m}{2} \right\rceil - 1 = \frac{m+1}{2} - 1 = \frac{m-1}{2}$$

Thus the claim is satisfied.

14. Suppose we have a B-tree of order $m \geq 4$. Suppose $b > 1$ and the number of keys at time t is $k(t) = b^t$. Prove mathematically (think derivative) that as $t \to \infty$ the minimum height approaches a line and find the slope of that line.

Solution:
The minimum height is given by:

$$\begin{aligned} h(t) &= \log_m(k+1) - 1 \\ &= \log_m(b^t + 1) - 1 \\ &= \frac{\ln(b^t+1)}{\ln m} - 1 \end{aligned}$$

Thus:

$$h'(t) = \frac{1}{\ln m} \frac{b^t \ln b}{b^t + 1}$$

Then we have:

$$\lim_{t \to \infty} \frac{1}{\ln m} \frac{b^t \ln b}{b^t + 1} = \frac{1}{\ln m} \ln b = \log_m b$$

This is then the slope of that line.

15. We know that for a B-tree of order $m \geq 3$, height h, and k keys, that:

$$2\lceil m/2 \rceil^h - 1 \leq k \leq m^{h+1} - 1$$

Suppose m and h are fixed. Prove that for every k in that range, we can actually construct a B-tree with k keys.

Note: The theorems which give us the inequalities above simply give us upper and lower bounds on k and say that those upper and lower bounds are possible, they don't say that all k in that range are possible - that's what you're proving here.

Solution:
The proof is by induction on k. For $k = 2\lceil m/2 \rceil^h - 1$ the sparsest possible B-tree will work.
Assume now that for some k with $2\lceil m/2 \rceil^h - 1 \leq k \leq m^{h+1} - 2$ (one less than the densest) we can build a B-tree with k keys.
First build such a B-tree. Now we have to see how we can add an extra key (for a total of $k+1$) without changing h.
Pick a lowest (closest to a leaf, or even a leaf) possible node (call it N) which is not full. We know such a node exists because this is not the densest possible B-tree. Pick any leaf node in the subtree rooted at N and insert a key which will end up in that leaf node. If N is a leaf we are done. Otherwise since the leaf node is full it will split, sending a key to the parent. If the parent is N then we are done, otherwise the parent is full and splits as well, sending a key to its parent. This process continues until N gains a key and we are done.

16. Suppose we have a B-tree of unknown order m and height $h = 1$ which has the property that any insertion requires exactly one split to balance. What can we conclude about the number of keys in the tree? Explain.

Solution:
In order to require exactly one split all the leaves must be full but the root cannot be. Thus the root has $m - 2$ or fewer keys and $m - 1$ or fewer children and so in total there are at most $m - 2 + (m - 1)(m - 1) = m - 2 + m^2 - 2m + 1 = m^2 - m - 1$ keys.

17. Suppose we have a B-tree of order $m \geq 3$ where m is odd. Prove that when we demote and merge due to a deletion that the resulting node is not overfull.

Solution:
We demote and merge due to a node being underfull, meaning it has $\lceil m/2 \rceil - 2$ keys, and we merge only with an adjacent sibling which has the bare minimum, meaning it has $\lceil m/2 \rceil - 1$ keys. Together, along with the demoted key, the number of keys is:

$$\left\lceil \frac{m}{2} \right\rceil - 2 + \left\lceil \frac{m}{2} \right\rceil - 1 + 1 = \frac{m+1}{2} - 2 + \frac{m+1}{2} - 1 + 1 = m - 1$$

18. Fixing both $m \geq 3$ and $h \geq 1$, suppose a B-tree of order m with height h has exactly one fewer than the maximum number of keys.

    (a) Prove that some leaf has exactly one fewer than the maximum number of keys.

    Solution:
    We know that exactly one node has fewer than the maximum number of keys. Suppose that node is not a leaf. If we pick a leaf in the subtree rooted at that node and insert a key into that leaf then this will generate a sequence of splits and promotions until that node gains a key and becomes full without increasing the height of the tree. Meanwhile the leaf we inserted the key into has split and the two new leaves each have room for at least one more key, contradicting the initial assumption.

    (b) Fixing both $m \geq 3$ and $h \geq 1$, suppose a B-tree or order m with height h has exactly one fewer than the maximum number of keys. Suppose that a leaf is chosen at random and a key is inserted into that leaf. What is the probability that the tree does not gain height?

    Solution:
    We know from (a) that exactly one leaf has one fewer than the maximum number of keys. The tree will not gain height if and only if we insert our key into that specific leaf or its neighbor(s).
    There are m^h leaves; if an end-leaf is the one with fewer than the maximum then the probability is $2/m^h$ and otherwise it is $3/m^h$.

19. Suppose b_h equals the number of different B-tree tree structures with order $m = 3$ and height h. We don't care about the specific keys, just the overall structure. Note that when counting structures, nodes containing different numbers of keys count towards different structures.

(a) It's true that $b_0 = 2$. Draw the two possible trees.

Solution:
The first is just a single node with one key and the second is just a single node with two keys.

(b) Prove that for $h \geq 1$ we have:

$$b_h = b_{h-1}^2 \left(b_{h-1} + 1\right)$$

Solution:
For a B-tree of order $m = 3$ with height h either:

- The root has one key in which case there are two child subtrees each with height $h-1$ and there are b_{h-1} choices for each, giving a total of b_{h-1}^2 possibilities.
- The root has two keys in which case the three child subtrees are height $h-1$ and there are b_{h-1} choices for each, giving a total of b_{h-1}^3 possibilities.

The total is then:

$$b_h = b_{h-1}^2 + b_{h-1}^3 = b_{h-1}^2 \left(b_{h-1} + 1\right)$$

20. For each of the following m and h calculate the minimum and maximum number of keys in a B-tree of order m and height h. Simplify.

m	h	Min	Max
3	2	7	26
4	3	15	255

We have:

- For $m = 3$ and $h = 2$ the minimum is $2\lceil 3/2 \rceil^2 - 1 = 8 - 1 = 7$.
- For $m = 3$ and $h = 2$ the maximum is $3^{2+1} - 1 = 26$.
- For $m = 4$ and $h = 3$ the minimum is $2\lceil 4/2 \rceil^3 - 1 = 16 - 1 = 15$.
- For $m = 4$ and $h = 3$ the maximum is $4^{3+1} - 1 = 255$.

21. Consider the following B-tree of order $m = 3$:

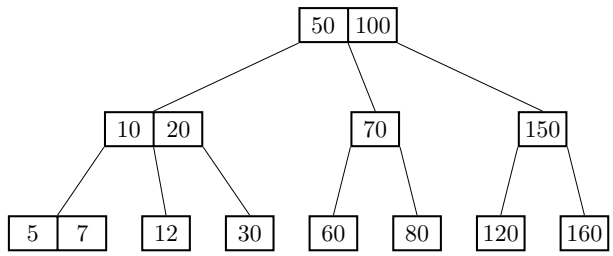

Figure 4: A B-Tree

For each of the following operations indicate the number of right key rotations, left key rotations, split-and-promotions, and demote-and-merges that each command would cause.

Operation	RKR	LKR	S&P	D&M
`insert(13)`	0	0	0	0
`insert(8)`	1	0	0	0
`delete(50)` using IOP	0	0	0	1
`delete(50)` using IOS	1	0	0	1
`delete(100)` using IOS	0	0	0	2

We have:
`insert(13)`: The 13 goes into the leaf with the 12 and we are done.
`insert(8)`: The 8 goes into the leaf with the $5, 7$ which is now overfull but a right rotation fixes it and we are done.
`delete(50)` using IOP: The IOP is 30. When we delete 30 that node is empty and we can't rotate so we demote 20 and merge with 12 and we are done.
`delete(50)` using IOS: The IOS is 60. When we delete 60 that node is empty and we can't rotate so we demote 70 and merge with 80. Then the node where 70 was is now empty but we can right rotate and we are done.
`delete(100)` using IOS: The IOS is 120. When we delete 120 that node is empty and we can't rotate so we demote 150 and merge with 160. Then the node where 150 was is now empty and we can't rotate so we demote 100 and merge with 70 and we are done.

22. Suppose we have a B-tree of order m where m is even. Prove using key counts that demote-and-merge does not yield an overfull node. Explain your calculation with sentences.

Solution:

We demote-and-merge when our node is underfull and the adjacent siblings have the minimum number of keys. Thus we are merging a node with $\lceil m/2 \rceil - 2$ keys and a node with $\lceil m/2 \rceil - 1$ keys. Along with the demoted key the total number of keys in our new node will be:

$$\lceil m/2 \rceil - 2 + \lceil m/2 \rceil - 1 + 1 = m - 2$$

Since this is not more than $m - 1$ the node is not overfull.

23. Suppose we are looking at a specific B-tree (meaning you don't know what it looks like but you can't specify what it looks like) with order $m \geq 3$ and height h which contains one fewer than the maximum number of keys. Explain how we could choose a key to insert which would not increase the height and why it would not increase the height.

Solution:

Since the tree is not at maximum there is a node with fewer than $m - 1$ keys. Pick a leaf node in the subtree rooted at that node and insert a key which ends up in that leaf. Rotations will not change the height and any splitting which propagates up the tree will end up at that node which will not be overfull and the splitting will stop. This will result in a tree with one more key and the same height.

Note: It is true that the node with $m - 1$ keys must be in a leaf but you can't say that without proving it. That proof might go something like the following:
If the node with $m - 1$ keys were not a leaf then it would be an internal node. We could add a key to that node which would then force us to add another child which would then introduce more keys without increasing the height, meaning we did not have one fewer than the maximum number of keys. Thus the node with $m - 1$ keys is a leaf and we simply insert a key into that leaf.